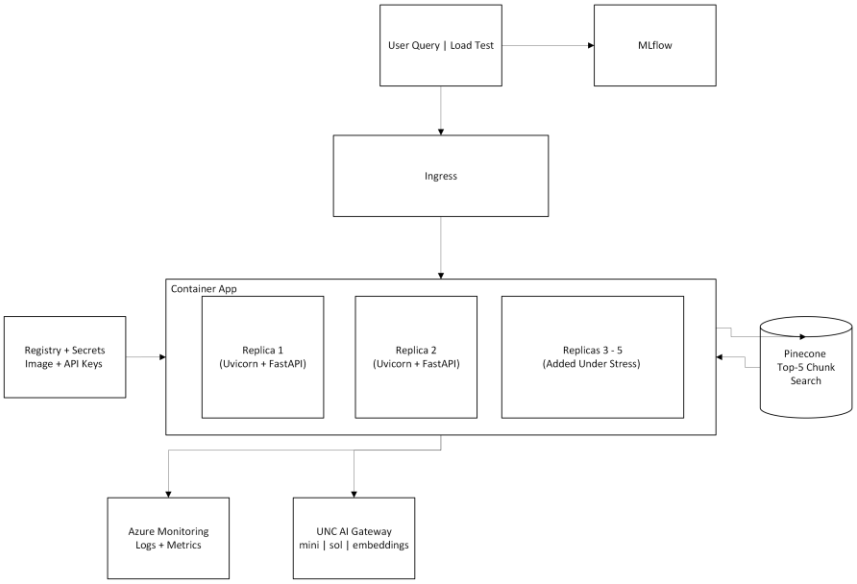
Cloud Deployment Report: Warhammer 40K Rules Assistant

DATA 790 · Joshua Farmer · Live API: https://w40k-rag.victorioushill-f2ce5271.eastus.azurecontainerapps.io/docs

1. What I deployed

My Milestone 2 security-hardened RAG assistant, turned from a notebook into a FastAPI service (`POST /ask`, `GET /health`, `GET /metrics`).

- **Container:** packaged with Docker.
- **Cloud:** deployed to Azure Container Apps, with HTTP autoscaling.
- **Kubernetes:** the same image also runs from Kubernetes manifests.
- **Monitoring:** Azure Monitor and MLflow.



Architecture

How a question flows:

1. The ingress hands the request to one of 1–5 identical, stateless replicas.
2. The replica runs the guard (gpt-4.1-mini) and embeds the question (text-embedding-3-small) through the UNC AI Gateway.
3. It pulls the top 5 chunks from Pinecone.
4. It has gpt-5.6-sol write the answer from those chunks.
5. It runs the output guard and returns the answer.

The gateway supplies the models, Pinecone supplies the knowledge, and the replica runs the steps. Keys live in Container App secrets, never in the image.

2. Deployment

- **Docker:** `python:3.12-slim`, pinned requirements, uvicorn on port 8000. `.dockerignore` keeps `.env` out of the image.
- **Azure:** `az containerapp up --source .` built the image in the cloud and created the registry, the environment and the Log Analytics workspace. Keys were added as secrets and mapped with `secretref:` env vars.

- **Kubernetes** (Docker Desktop, kubeadm): `k8s/deployment.yaml` (image, secret, `/health` readiness probe, CPU/memory requests), `service.yaml` (LoadBalancer on port 80 → 8000) and `hpa.yaml` (1–5 pods at 70% CPU, with metrics-server). Kubernetes pieces map to Container Apps like this: Deployment = revision, Pod = replica, Service = ingress, HPA = KEDA scale rule.

```
PS C:\Users\joshu\Desktop\Data_790_AdvAI_Frameworks\Project\cloud> kubectl get pods,svc,hpa
NAME                                READY   STATUS    RESTARTS   AGE
pod/w40k-rag-8549db76fc-sz5d8   1/1     Running   0          122m

NAME                 TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
service/kubernetes   ClusterIP      10.96.0.1       <none>        443/TCP        128m
service/w40k-rag     LoadBalancer   10.111.171.25   localhost     80:32748/TCP   122m

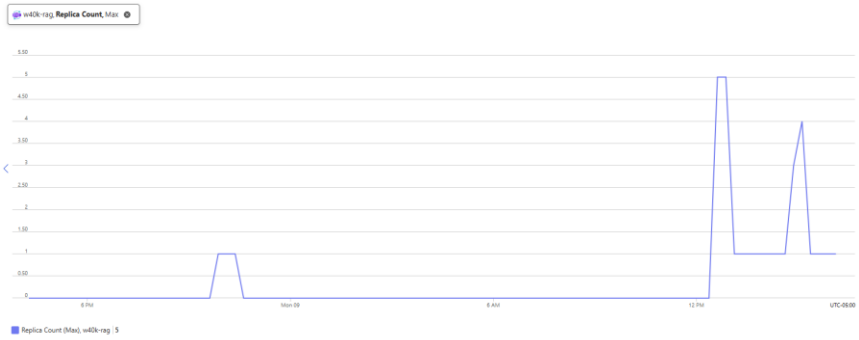
NAME                                                  REFERENCE             TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/w40k-rag   Deployment/w40k-rag   cpu: 0%/70%   1         5         1          122m
```

kubectl get pods,svc,hpa

3. Autoscaling and load test

The Azure scale rule: 1–5 replicas, adding one when there are more than 5 concurrent requests per replica, with a 300 s cooldown before scaling down. The load test sent 100 × `POST /ask` at 10 concurrent.

Result	Value
Success rate	100% (no 5xx errors; 408 × 2xx over the day)
Avg / p95 latency	2.67 s / 3.68 s
Peak replicas	5 (first test), 4 (MLflow run), back to 1 after cooldown



Replica count



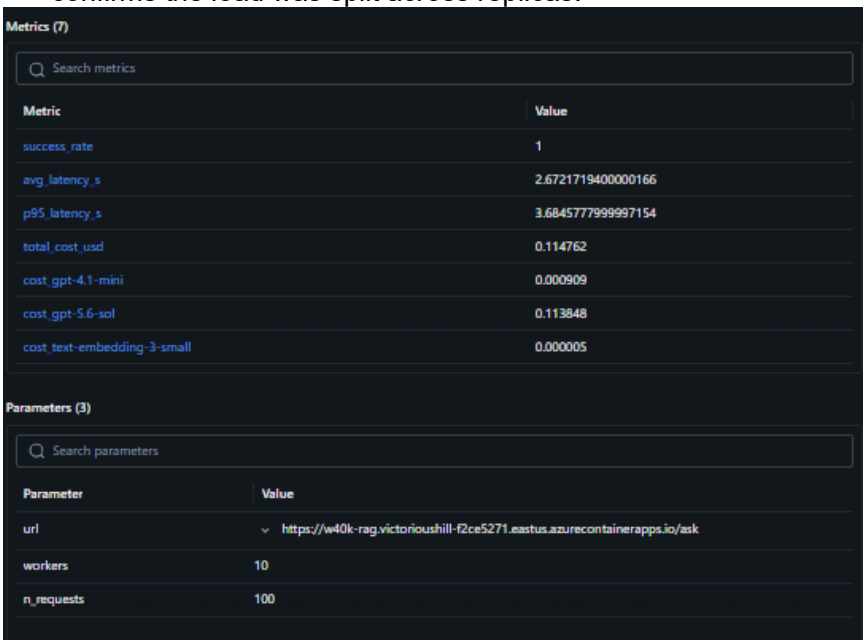
Requests by status code

Takeaways:

- **The upstream LLM is the real bottleneck.** A first try at 20 concurrent timed out because the UNC gateway throttled the slow reasoning-model calls. Every replica shares that quota, so scaling the app can't raise it.
- **CPU is the wrong scaling signal here.** CPU sat near 0% even under load, because the app spends its time waiting on the network. That's why Azure scales on HTTP concurrency instead of CPU.

4. Monitoring and observability

- **Azure Monitor** covers infrastructure: replica count, requests by status code, and response time. Log Analytics holds the container logs.
- **MLflow** logs every load-test run: params (URL, workers, number of requests) and metrics (success rate, latency, cost per model).
- **App `/metrics`** shows cost by model and content-filter flags. It is per replica, because the tracker lives in memory. The 100-request MLflow run showed only $0.11 of the expected ≈ $0.34, which confirms the load was split across replicas.



MLflow run

5. Cost analysis and optimization

Model	Role	Cost / 100 questions	Share
gpt-5.6-sol	answer	$0.333	99.1%
gpt-4.1-mini	guard	$0.003	0.8%
text-embedding-3-small	retrieval	$0.00002	<0.1%
Total		$0.336 (≈ $0.0034 / question)	

At 1,000 questions/day that's about $100/month in LLM cost. Infrastructure (the Container Apps replicas, the registry and Log Analytics) costs far less than the LLM calls.

Done:

- Model tiering: the cheap model handles the guard, and only the answer uses the reasoning model.
- Attacks are blocked before the expensive generation step.
- Scaling is on concurrency, not CPU.

6. Takeaways

- **One call drives everything.** Answer generation is about 99% of the cost and most of the latency. Which model answers matters far more than the infrastructure.
- **Scaling has a ceiling.** More replicas stop requests from piling up, but every replica shares the same LLM gateway and rate limit.
- **Match the scaling signal to the workload.** This app waits on the network, so it scales on HTTP concurrency, not CPU.
- **In-memory metrics don't survive scale-out.** Use the platform's monitoring (Azure Monitor) for totals.